

III. Tableau de Bord — Conception Backend/Frontend

III.1 Choix technologique : la stack MERN

III.1.1 Présentation de la stack

La plateforme EmissionsIQ repose sur une **stack MERN** : MongoDB, Express.js, React et Node.js. Ce choix technologique couvre l'intégralité du système de façon homogène, du stockage des données jusqu'à l'interface utilisateur, avec un seul langage — JavaScript/TypeScript — traversant toutes les couches.

L'unité de langage n'est pas un argument cosmétique : elle réduit le coût cognitif du projet (un seul écosystème de dépendances, un seul gestionnaire de paquets npm, une seule syntaxe asynchrone basée sur `async/await`), facilite le partage de types de données entre le backend et le frontend, et simplifie le déploiement sur un environnement industriel contraint.

III.1.2 Étude comparative

Le tableau suivant confronte la stack MERN aux deux alternatives les plus courantes dans les projets de supervision industrielle : Django/PostgreSQL et Spring Boot/MySQL.

Critère	MERN (retenu)	Django + PostgreSQL	Spring Boot + MySQL
Langage	JavaScript / TypeScript	Python + JavaScript	Java + JavaScript
Modèle d'I/O	Non bloquant (event loop)	Synchrone (WSGI)	Multi-thread (JVM)
Données temps réel (WebSocket/MQTT)	Natif — <code>ws</code> , <code>mqtt</code> bibliothèques Node.js	Django Channels (complexité sup.)	Spring WebSocket (verbeux)
Flexibilité du schéma	MongoDB — schéma souple, évolutif	PostgreSQL — schéma rigide, migrations	MySQL — schéma rigide, migrations
Montée en charge MQTT (172 k+ lectures/30 j)	Event loop Node.js — ingestion non bloquante	GIL Python — scalabilité limitée	JVM — overhead mémoire élevé
Courbe d'apprentissage PFE	Faible — écosystème unifié JS	Moyenne — deux langages	Élevée — Java + Spring
Disponibilité locale (Sfax)	Forte — large communauté	Forte	Faible
Production de rapports PDF	Puppeteer (npm) — sans dépendance externe	WeasyPrint / ReportLab	iText / JasperReports
Couplage frontend/backend	Fort — TypeScript partagé, Zod, DTOs	Faible — sérialisation DRF	Faible — sérialisation Jackson

Justification du choix MERN. Le projet présente deux contraintes techniques dominantes qui orientent le choix :

1. **L'ingestion MQTT temps réel** : le backend reçoit en continu des lectures publiées par les nœuds ESP32 sur le broker Mosquitto. Chaque lecture déclenche une chaîne synchrone — validation, écriture MongoDB, moteur d'alertes, diffusion WebSocket. L'architecture event loop de Node.js, non bloquante par nature, absorbe ces rafales sans créer de files d'attente. Un serveur Django synchrone (WSGI) devrait multiplier les workers pour obtenir le même résultat.
2. **La richesse des données capteurs** : les 6 polluants surveillés (CO_2 , NO_x , SO_2 , $\text{PM}_{2.5}$, COV, température/humidité) ont des schémas différents selon les capteurs, les zones et les configurations réglementaires. MongoDB permet de stocker ces variations sans migration de schéma, ce qui est critique dans un projet évolutif. Le modèle `ThresholdConfig` illustre cette souplesse : il supporte des

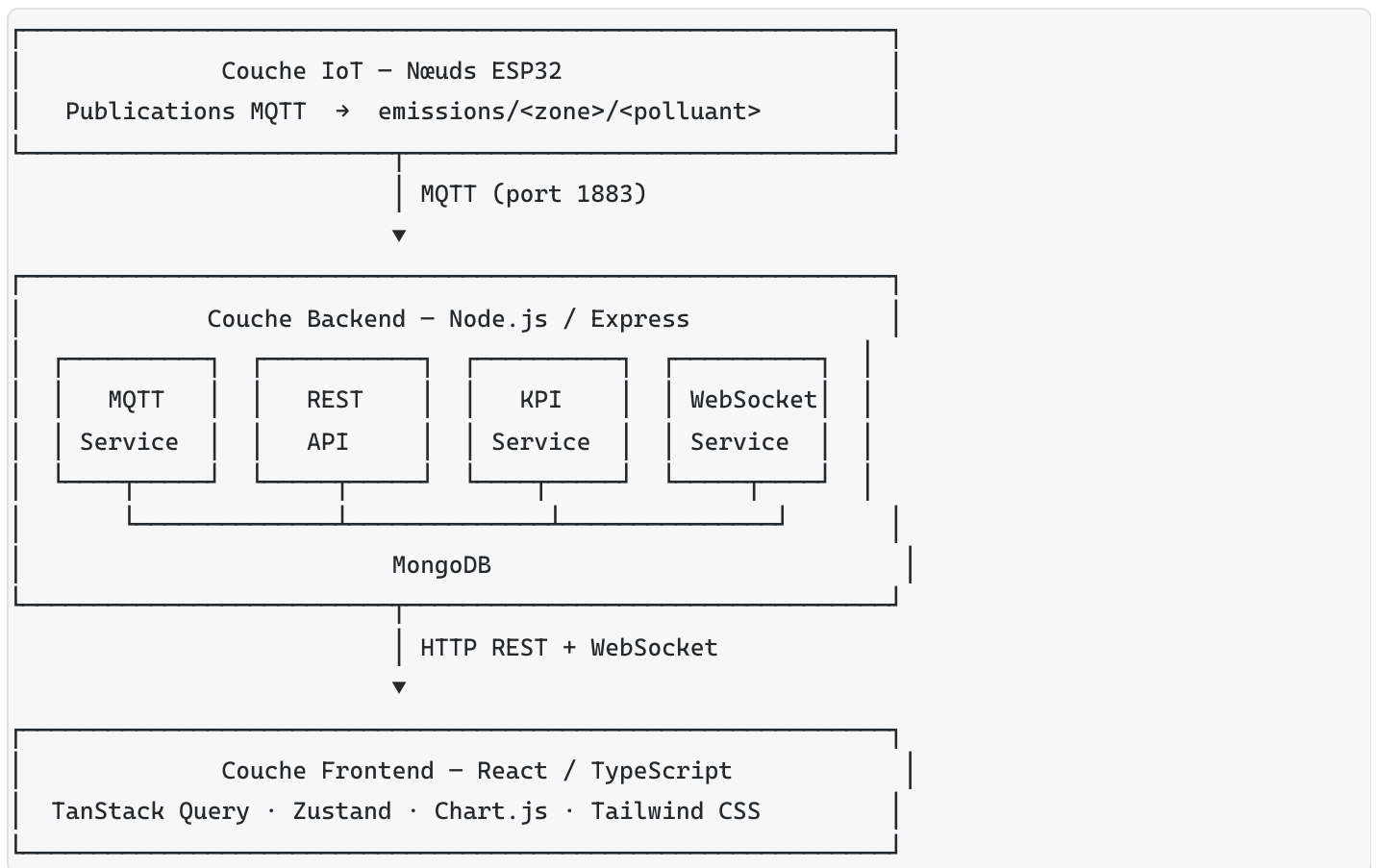
configurations réglementaires génériques (Annexe 1) et sectorielles (cimenteries, raffineries) dans le même document.

Conclusion : La stack MERN n'est pas choisie par défaut mais par adéquation aux contraintes du projet. Django aurait pu convenir pour un système à prédominance analytique ; Spring Boot pour un système à fort besoin transactionnel. Ici, la prédominance de l'ingestion temps réel et de la diffusion push justifie Node.js.

III.2 Architecture générale du système

III.2.1 Vue d'ensemble

L'architecture est organisée en **trois couches communicantes** :



III.2.2 Architecture backend en couches

Le backend suit un pattern **Routes** → **Controllers** → **Services** → **Repositories** → **Models**, qui isole chaque responsabilité :

Couche	Responsabilité	Exemple
Routes	Déclarer les endpoints HTTP et appliquer les middlewares	<code>alertRoutes.js</code> — <code>POST /:id/acknowledge</code>
Controllers	Désérialiser la requête HTTP, déléguer au service, sérialiser la réponse	<code>alertController.js</code>
Services	Logique métier — calculs KPI, moteur d'alertes, génération rapports	<code>KPIService.js</code> , <code>ReadingService.js</code>
Repositories	Abstraction des requêtes MongoDB (agrégations, filtres)	<code>ReadingRepository.js</code>
Models	Schémas Mongoose, hooks, méthodes d'instance	<code>Alert.js</code> , <code>AggregateData.js</code>

Cette séparation permet de tester chaque couche indépendamment et de substituer MongoDB par un autre moteur sans toucher aux services.

Séquence de démarrage du serveur :

- ① Connexion MongoDB (pool 200 connexions)
- ② Enregistrement des routes (14 fichiers)
- ③ Démarrage HTTP server (port 5000)
- ④ MQTT Service → abonnement `emissions/#`
- ⑤ WebSocket Server → écoute `/ws`
- ⑥ KPI Broadcaster → push toutes les 5 s
- ⑦ KPI Scheduler → cron `horaire/journalier/mensuel`

L'ordre est délibéré : la connexion MongoDB est établie avant tout abonnement MQTT pour éviter une avalanche de requêtes en file d'attente lors de l'ouverture de la connexion.

III.2.3 Architecture frontend

Le frontend est une **Single Page Application (SPA)** construite avec React 18 et TypeScript. L'arborescence des providers définit la portée des contextes globaux :

```

ErrorBoundary
├── QueryClientProvider (TanStack Query)
│   └── ToastProvider
│       └── BrowserRouter
│           └── WebSocketProvider
│               └── AppRoutes (lazy-loaded pages)

```

Chaque couche apporte une responsabilité précise :

- `QueryClientProvider` : cache des requêtes HTTP (30 s par défaut), invalidation sur événement WebSocket
 - `WebSocketProvider` : connexion persistante `ws://backend:5000/ws` , propagation des mises à jour temps réel via `applyWSMessage(queryClient, msg)`
 - `AppRoutes` : routage protégé par permission (composant `ProtectedRoute`)
-

III.3 Architecture frontend — Conception détaillée

III.3.1 Stack technologique et justification des choix

Le frontend est une **Single Page Application (SPA)** construite avec React 18 et TypeScript. Chaque bibliothèque a été retenue pour une raison technique précise.

Technologie	Version	Rôle	Justification
React 18	18.3	Framework UI	Composants réutilisables, Concurrent Rendering, Suspense natif
TypeScript	5.3	Typage statique	Détection d'erreurs à la compilation, contrat d'interface entre features
Vite	5.0	Bundler	HMR en < 50 ms, build optimisé (code splitting automatique)
React Router	6.22	Routage	Navigation SPA sans rechargement, <code>lazy()</code> pour le code splitting par page
TanStack Query	5.17	État serveur	Cache 30 s, invalidation ciblée, retry automatique, <code>staleTime/gcTime</code>
Zustand	4.5	État UI global	Store léger sans boilerplate Redux, sélecteurs réactifs
Chart.js 4	4.4	Visualisation	Canvas 2D performant, interactivité native, plugins Tailwind
Tailwind CSS	3.4	Styling	Classes utilitaires, design system cohérent, purge CSS automatique
React Hook Form	7.49	Formulaires	Validation en temps réel, zéro re-render inutile
Zod	3.22	Validation schémas	Schemas TypeScript-first partagés formulaires/API
Axios	1.6	Client HTTP	Intercepteurs pour refresh token, baseURL centralisée
Lucide React	0.331	Icônes	SVG tree-shakeable, cohérence visuelle

Choix de TanStack Query vs Redux Toolkit Query. TanStack Query est retenu car le projet n'a pas besoin d'un store Redux global pour les données serveur — la séparation entre **état serveur** (TanStack Query) et **état UI** (Zustand) est plus claire et moins verbeux que RTK Query.

Choix de Zustand vs Context API. Le Context API React est rejeté pour les stores globaux à haute fréquence de mise à jour (sélection site/zone/période) car chaque mise à jour re-rend tous les consumers. Zustand utilise des sélecteurs réactifs qui ne re-rendent que les composants abonnés au champ modifié.

III.3.2 Structure des dossiers — Architecture Feature-First

Le frontend adopte une organisation **feature-first** : chaque fonctionnalité métier est co-localisée dans un dossier `features/`, contenant ses propres types, appels API, et hooks.

frontend/src/

App.tsx	← Root – configuration des providers
main.tsx	← Point d'entrée Vite
pages/	← Une page = une route
Overview/	← Tableau de bord principal
Alerts/	← Gestion des alertes
History/	← Historique des mesures
Compliance/	← Conformité réglementaire
AIPredictions/	← Prévisions et anomalies IA
Reports/	← Génération et téléchargement rapports
Config/	← Configuration KPI et seuils
Users/	← Gestion utilisateurs (SUPER_ADMIN)
Sites/ Zones/	← Gestion sites et zones
Industries/	← Gestion industries
Approvals/	← Workflow approbation
Team/ Operators/	← Gestion équipes
Login/ Register/	← Authentification
features/	← Modules métier (API + hooks + types)
alerts/	
api/	← alertsApi.ts – appels HTTP
hooks/	← useAlerts, useAlertStats, useAlertScope
types/	← Alert, AlertFilters, AlertSeverity
auth/	
api/	← authApi.ts
hooks/	← useAuth (JWT + user profile)
store/	← zoneStore (zone sélectionnée)
utils/	← token storage
kpi/	
api/	← kpiApi.ts
hooks/	← useKPISummary, useKPIHistory, useKPIConfig
utils/	← kpiCalculations, mtdSeries
readings/	
hooks/	← useLatestReadings
reports/	
hooks/	← useReports, useGenerateReport
websocket/	
websocketClient.ts	← Classe WebSocketClient (reconnexion, heartbeat)
WebSocketProvider.tsx	← Provider React
applyWSMessage.ts	← Invalidation ciblée TanStack Query
useWebSocketSubscription.ts	
ia/ sites/ zones/ users/ config/	

components/	← Composants partagés
— layout/	← MainLayout, Sidebar, Topbar, PageHeader
— charts/	← IPEGauge, HistoryChart, MTDTrendChart
— kpi/	← KPICard, PollutantCard, ComplianceTable
— alerts/	← AlertList, AlertItem, AlertFilters, AlertDetailModal
— common/	← ProtectedRoute, ZoneSwitcher, PermissionGate
— ui/	← Badge, Button, Card, Modal, Table, Toast
store/	
— selectionStore.ts	← siteId, zoneId, period (global)
— uiStore.ts	← sidebar collapsed, mobile menu
lib/	
— api/	
— axios.ts	← Intercepteur refresh token
— endpoints.ts	← Centralisation des URLs
— queryClient.ts	← Config TanStack Query + queryKeys
— constants/	
— roles.ts	← Enum Role, ROLE_PERMISSIONS
— pollutants.ts	← POLLUTANT_CODES, labels, couleurs
— tunisiaDecret.ts	← Limites VLE Décret 2010-2519
— kpiTargets.ts	← Cibles KPI (TD ≤2%, IPE ≥95...)
— rbac/	
— checkPermission.ts	← hasPermission, canAccessResource
— permissions.ts	
— utils/	
— formatters.ts	← formatNumber, formatDate (locale fr)
— colorUtils.ts	← statusFromRange (badge couleur)

III.3.3 Gestion de l'état — Architecture à deux stores

L'état frontend est segmenté en **deux niveaux distincts** selon la nature des données :

Niveau 1 — État serveur (TanStack Query). Toutes les données provenant de l'API REST sont gérées par TanStack Query. Le `queryClient` est configuré avec :

- `staleTime: 30 000 ms` — les données restent fraîches 30 secondes sans refetch
- `gcTime: 300 000 ms` — les entrées inutilisées sont supprimées du cache après 5 minutes
- `retry : 3 tentatives avec backoff exponentiel (1s → 2s → 4s)`, sauf pour les erreurs 4xx (permission refusée) — pas de retry
- `refetchOnWindowFocus: false` — pas de refetch automatique au focus (données industrielles stables)

```

queryKeys = {
  kpi:      { summary, history(pollutantId), ipe, td, config }
  alerts:   { list(filters), stats, detail(id) }
  readings: { latest, list }
  reports:  { list, detail(id) }
  ia:       { forecast(zoneId), anomalies(zoneId) }
  sites:    { list, detail(id) }
  zones:    { list, detail(id) }
  ...
}

```

Niveau 2 — État UI (Zustand). Les états purement UI sans persistance serveur sont gérés par deux stores Zustand :

```

selectionStore :
  siteId      : string | null ← site sélectionné globalement
  zoneId      : string | null ← zone sélectionnée globalement
  sensorNodeId : string | null
  period      : 'hour'|'day'|'week'|'month'|'year'
  → Partagé par Overview, Alerts, History, Compliance, AI

uiStore :
  sidebarCollapsed : boolean
  mobileMenuOpen   : boolean

```

Pourquoi cette séparation ? TanStack Query gère la synchronisation avec le serveur (cache, revalidation, mutations). Zustand gère uniquement la sélection de contexte et l'UI — il n'y a jamais de doublon entre les deux.

III.3.4 Flux temps réel — Architecture WebSocket côté frontend

La connexion WebSocket est encapsulée dans la classe `WebSocketClient` qui gère l'ensemble du cycle de vie :

```

WebSocketClient
├── connect()           → ouvre ws://backend:5000/ws
├── authenticate()      → envoie {userId, role, email}
├── subscribe(topics)   → s'abonne aux topics KPI
├── onMessage(handler) → reçoit les broadcasts
├──
├── Reconnexion automatique :
│   ├── délai initial : 1 000 ms
│   ├── facteur exponentiel : ×2 à chaque échec
│   └── délai maximum : 30 000 ms
├──
└── Heartbeat : ping toutes les 30 s → maintien de la connexion

```

Flux de mise à jour de l'interface à la réception d'un message WebSocket :

```

WebSocketServer (Node.js)
  | broadcastKPIUpdate() ou broadcastAlert()
  ▼
WebSocketClient.onmessage
  | JSON.parse → WSMMessage
  ▼
WebSocketProvider → applyWSMessage(queryClient, msg)
  |
  |— msg.type === 'kpi_update'
  |   → queryClient.invalidateQueries(['kpi', 'summary'])
  |   → queryClient.invalidateQueries(['readings'])
  |
  |— msg.type === 'alert'
  |   → queryClient.invalidateQueries(['alerts'])

```

L'invalidation **ciblée** par queryKey est le mécanisme clé : elle ne déclenche un refetch que pour les composants actuellement montés qui utilisent la queryKey invalidée. Un composant sur la page Alerts recevra le nouveau fetch ; un composant sur la page Config ne sera pas affecté.

III.3.5 Contrôle d'accès frontend — Trois mécanismes

Mécanisme 1 — ProtectedRoute. Chaque route est enveloppée dans un composant `ProtectedRoute` qui vérifie l'authentification et les permissions avant de rendre la page :

```

<ProtectedRoute requires={['VIEW_ALERTS']}>
  <Alerts />
</ProtectedRoute>

```

Logique interne :

1. `isInitialized` ? sinon → `LoadingSpinner`
2. `isAuthenticated` ? sinon → `Navigate to /login`
3. `role` === `requiredRole` ? sinon → `Navigate to /unauthorized`
4. `hasAnyPermission(role, requires)` ? sinon → `Navigate to /unauthorized`
5. → `render children`

Mécanisme 2 — PermissionGate. Pour les éléments UI conditionnels (boutons, colonnes de tableau, actions), le composant `PermissionGate` rend ou masque ses enfants selon la permission :

```

<PermissionGate requires="RESOLVE_ALERT">
  <Button>Résoudre</Button>
</PermissionGate>
// Le bouton "Résoudre" est invisible pour un OPERATOR

```

Mécanisme 3 — Scope dynamique (ZoneSwitcher + useAlertScope). Le composant `ZoneSwitcher` adapte son comportement selon le rôle :

Rôle	Comportement ZoneSwitcher
OPERATOR	Liste déroulante des zones assignées (<code>user.zonesAssigned</code>)
SITE_SUPERVISOR	Zones regroupées par site assigné (fetch <code>zones/site/:id</code>)
HEAD_SUPERVISOR	Sélecteur site → reload zones du site sélectionné
SUPER_ADMIN / AUDITOR	Aucun sélecteur — accès global

La sélection est propagée dans `selectionStore` et toutes les pages consomment `siteId` et `zoneId` pour filtrer leurs requêtes API.

III.3.6 Pages et composants — Conception détaillée

Page Overview (`/overview`) — Tableau de bord principal

La page Overview est la plus complexe du système. Elle agrège **8 sources de données parallèles** via `useQueries` :

```

useKPISummary(params)      → TD, IPE, EMJ, RCO2 + deltas période précédente
useKPIConfig()              → airflow, poids polluants, cibles
useKPIHistory(pollutant)    → évolution journalière EMJ/IPE/RCO2
useLatestReadings(params)  → dernières lectures par polluant
useSites()                  → nombre de sites actifs
useAlerts({ pageSize: 5 }) → 5 dernières alertes ouvertes
useQueries([ ... TD par polluant ]) → TD journalier par polluant (optionnel)
useWebSocketSubscription() → abonnement topics kpi/alerts

```

La logique d’affichage est structurée en **panneaux interactifs** :

KPI Cards (4) – TD | IPE | EMJ | RCO₂

[clic → scroll vers detail panel]

| selectedKPI ∈ {TD, IPE, EMJ, RCO₂}



Detail Panel (conditionnel)

TD → MTDTrendChart mensuel + barres/polluant

IPE → IPEGauge + MTDTrendChart journalier

EMJ → sélecteur polluant + courbe cumulée

RCO₂ → courbe mensuelle + baseline vs actuel

PollutantCards (6 polluants + 2 env.)

[clic → focalise HistoryChart sur ce polluant]

HistoryChart (2/3)

multi-séries + VLE

AlertList (1/3)

5 alertes ouvertes

Interaction PollutantCard → HistoryChart. Un clic sur une `PollutantCard` appelle `focusPollutant(code)` qui : (1) filtre `filteredHistory` pour n'afficher que ce polluant, (2) fait défiler la page vers le graphique d'historique (`historyRef.scrollToView`), (3) met à jour `selectedMetric` pour surligner la carte active.

Page Alertes (/alerts) — Filtres URL synchronisés

La page Alertes utilise `useSearchParams` (React Router) pour persister les filtres dans l'URL. Cela permet de partager un lien filtré (`/alerts?severity=critical&status=open`) ou de retrouver les filtres après navigation :

URL params ↔ AlertFilters state

?severity=critical ↔ `filters.severity`

?status=open ↔ `filters.status`

?pollutant=NOX ↔ `filters.pollutant`

?page=2 ↔ `filters.page`

Le hook `useAlertScope()` enrichit automatiquement les filtres avec le périmètre RBAC de l'utilisateur connecté (siteld pour un SITE_SUPERVISOR, industryId pour un HEAD_SUPERVISOR), sans que la page ait besoin d'en avoir connaissance.

Système de graphiques — Chart.js

Quatre types de graphiques sont utilisés, configurés dans `chartSetup.ts` (enregistrement des composants Chart.js nécessaires) :

Composant	Graphique	Usage
<code>IPEGauge</code>	Jauge circulaire demi-cercle (Doughnut)	Score IPE 0–100 avec aiguille
<code>MTDTrendChart</code>	Barres + ligne cible	Tendance mensuelle KPI
<code>HistoryChart</code>	Lignes multi-séries + ligne seuil	Concentrations polluants vs VLE
<code>EnvironmentChart</code>	Lignes duales (T° et HR)	Contexte météorologique

Configuration de l'axe temporel. Chart.js est configuré avec `chartjs-adapter-date-fns` pour traiter les timestamps ISO 8601 nativement. Les labels sont formatés en `fr-FR` grâce à l'utilitaire `formatDate` (locale `date-fns`).

III.3.7 Gestion des formulaires et validation

Les formulaires utilisent **React Hook Form + Zod** selon un pattern uniforme :

```

Schéma Zod (lib/validation/)
  auth.schema.ts    → LoginSchema, RegisterSchema
  config.schema.ts  → KPIConfigSchema, ThresholdSchema

Formulaire (React Hook Form)
  const form = useForm<LoginInput>({
    resolver: zodResolver(LoginSchema)
  })
  → Validation déclenchée à chaque keystroke (mode 'onChange')
  → Erreurs affichées inline immédiatement
  → Soumission bloquée si schéma invalide
  
```

III.3.8 Performance et lazy loading

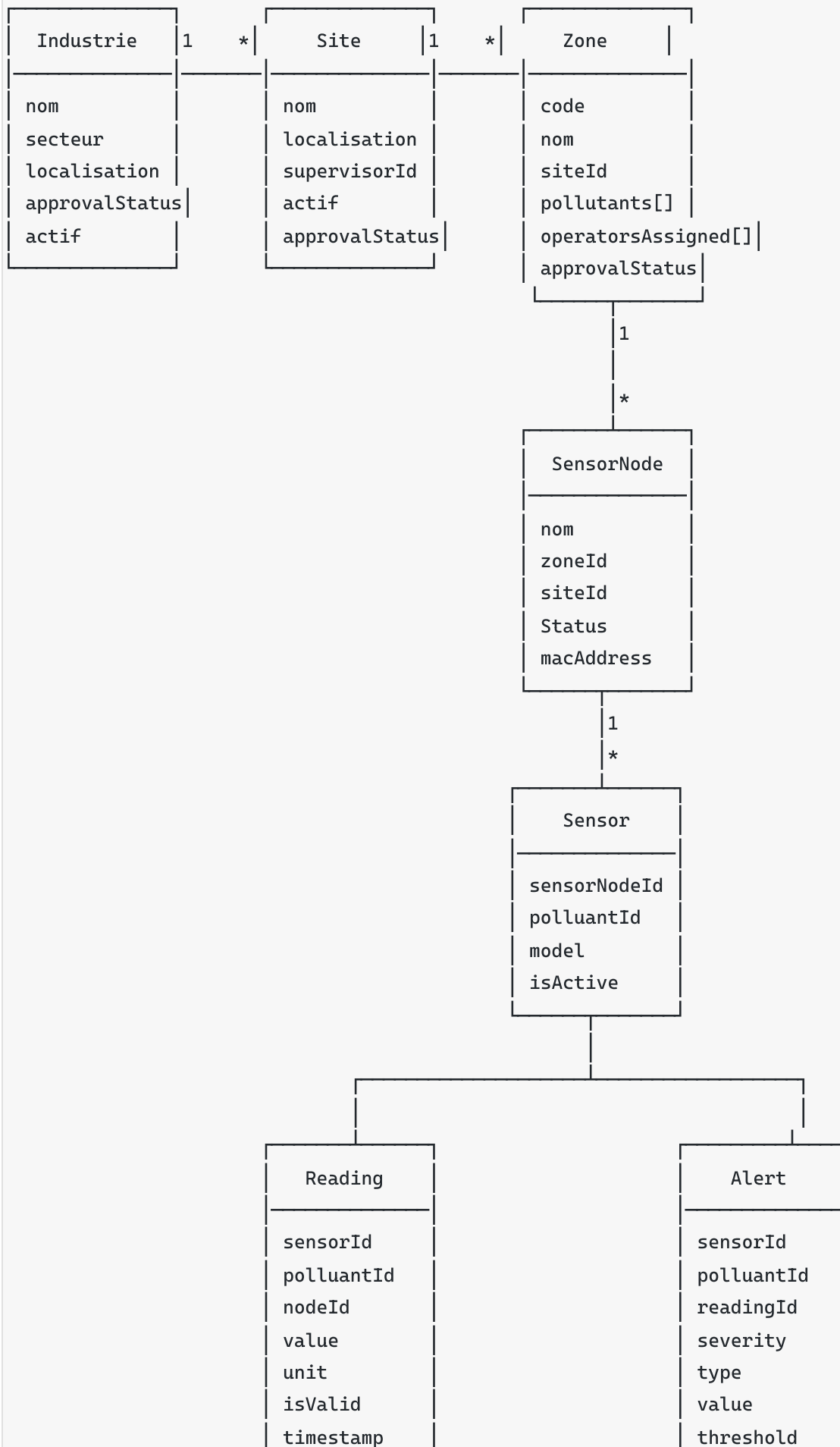
Toutes les pages sont **lazy-loaded** via `React.lazy()` : le bundle JavaScript de chaque page n'est téléchargé que lors de la première navigation vers cette page. Le composant `Suspense` affiche un `LoadingSpinner` pendant le téléchargement.

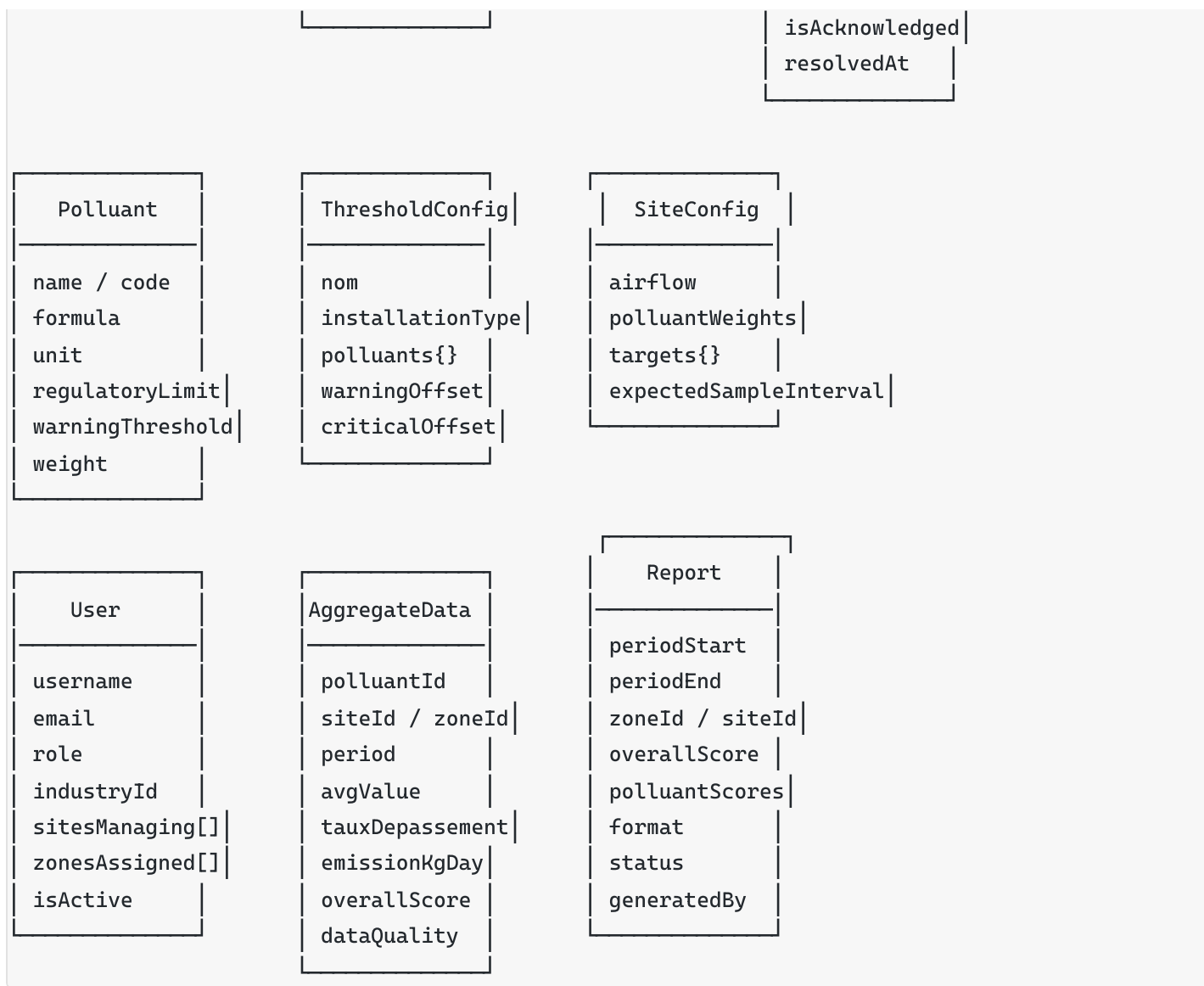
La barre de progression `StaleTime: 30s` signifie que les données d'une page récemment visitée sont affichées instantanément depuis le cache lors du retour sur cette page — aucun spinner visible pour l'utilisateur.

Les squelettes (`Skeleton`) sont affichés pendant les états de chargement initial pour éviter le layout shift (déplacement visuel lors de l'apparition des données).

III.4 Modèle de données — Diagramme de classes UML

Le modèle de données reflète la hiérarchie industrielle du système : une industrie possède des sites, chaque site contient des zones, chaque zone héberge des nœuds capteurs.

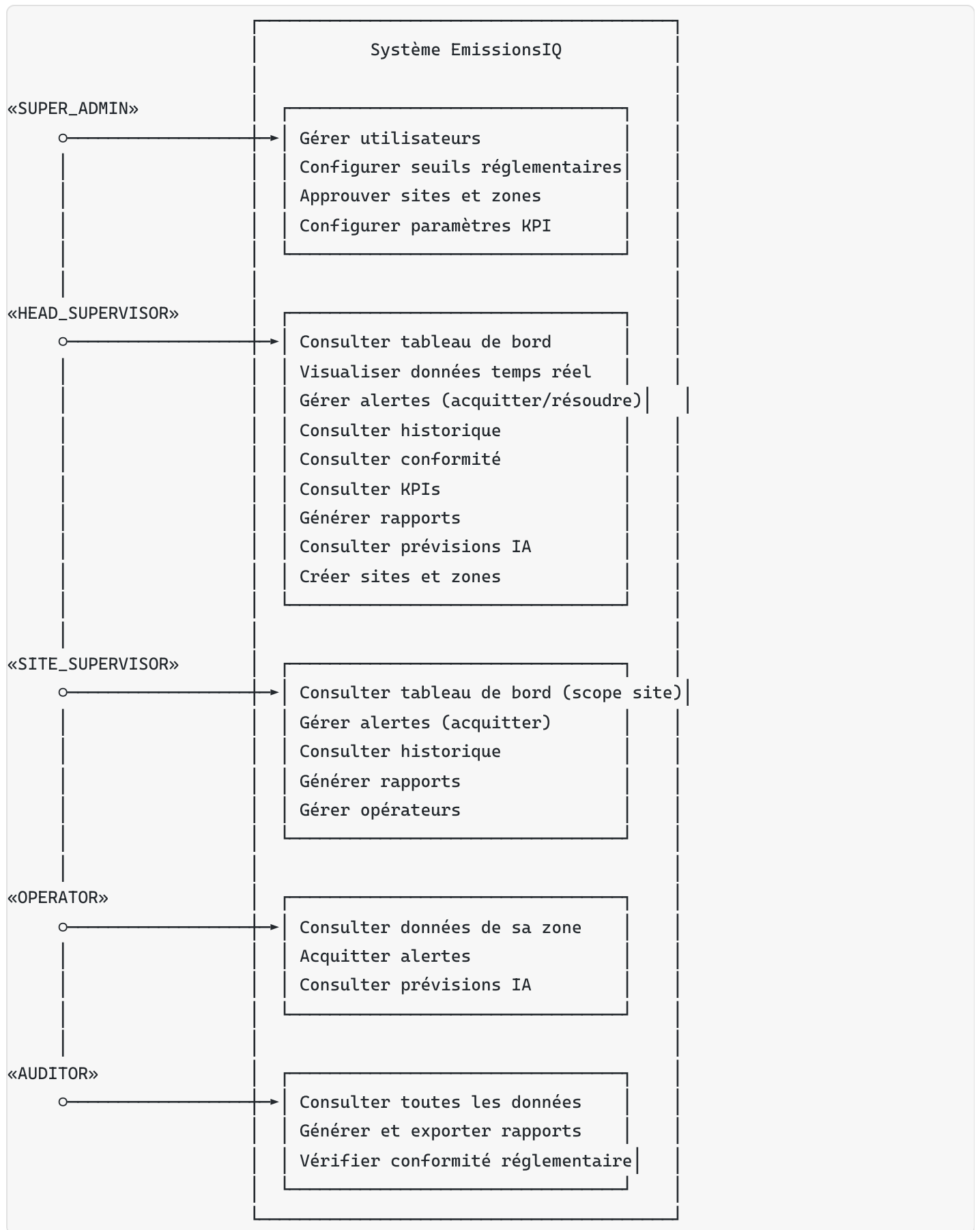




Dénormalisation stratégique. Le modèle `SensorNode` duplique `siteId` et `IndustrieId` bien qu'ils soient déjà accessibles via `Zone`. Cette dénormalisation évite des jointures en cascade lors des calculs KPI qui doivent filtrer les nœuds d'un site ou d'une industrie entière — une optimisation critique pour une collection `readings` de 172 000 documents par mois.

III.5 Diagramme de cas d'utilisation général

Le système définit cinq rôles avec des périmètres d'accès strictement délimités.

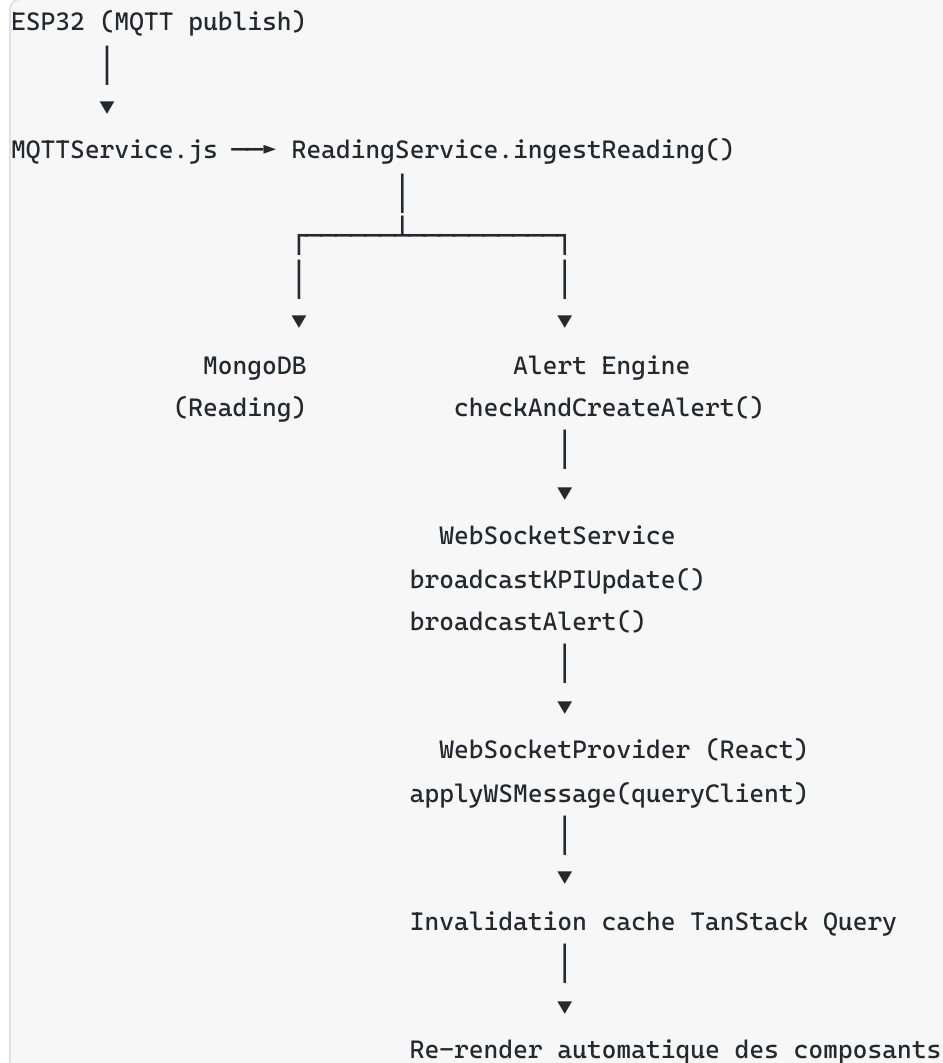


III.5 Modules fonctionnels clés — Conception détaillée

III.5.1 Module Visualisation temps réel

Le module de visualisation constitue le cœur du tableau de bord. Il repose sur un mécanisme de **mise à jour push** : le frontend ne poll pas le serveur mais reçoit les nouvelles données dès qu'elles sont disponibles.

Architecture du flux temps réel :



Le composant `WebSocketProvider` maintient une connexion persistante sur `/ws`. À chaque message reçu, `applyWSMessage` invalide sélectivement les caches TanStack Query concernés, provoquant un re-fetch ciblé plutôt qu'un rechargement global.

Composants de visualisation :

Composant	Type de graphique	Données affichées
IPE Gauge	Jauge circulaire (0–100)	Score IPE courant avec seuil cible
MTD Trend Chart	Barres + ligne	Émissions journalières du mois en cours
History Chart	Lignes multi-séries	Concentration par polluant + ligne VLE
KPI Cards	Sparklines + indicateur	Valeur KPI + tendance vs période précédente
Pollutant Cards	Badge coloré	Dernière valeur mesurée vs seuil

La bibliothèque Chart.js est configurée avec l’adaptateur `chartjs-adapter-date-fns` pour un axe temporel nativement géré en fuseau horaire.

III.5.2 Module Historique

Le module historique expose une **vue temporelle agrégée** des données capteurs. La conception distingue deux sources de données selon la granularité demandée :

- Pour les périodes longues (mois, trimestre) : interrogation de la collection `AggregateData` (données pré-calculées par les schedulers KPI)
- Pour les périodes courtes (< 48h) : interrogation directe de `Reading` via `buildLiveDailyHistory()` dans `KPIService`

Cette dualité est transparente pour le frontend : l’API `GET /api/kpi/history/:polluantId` renvoie toujours le même format de réponse, quel que soit le backend utilisé.

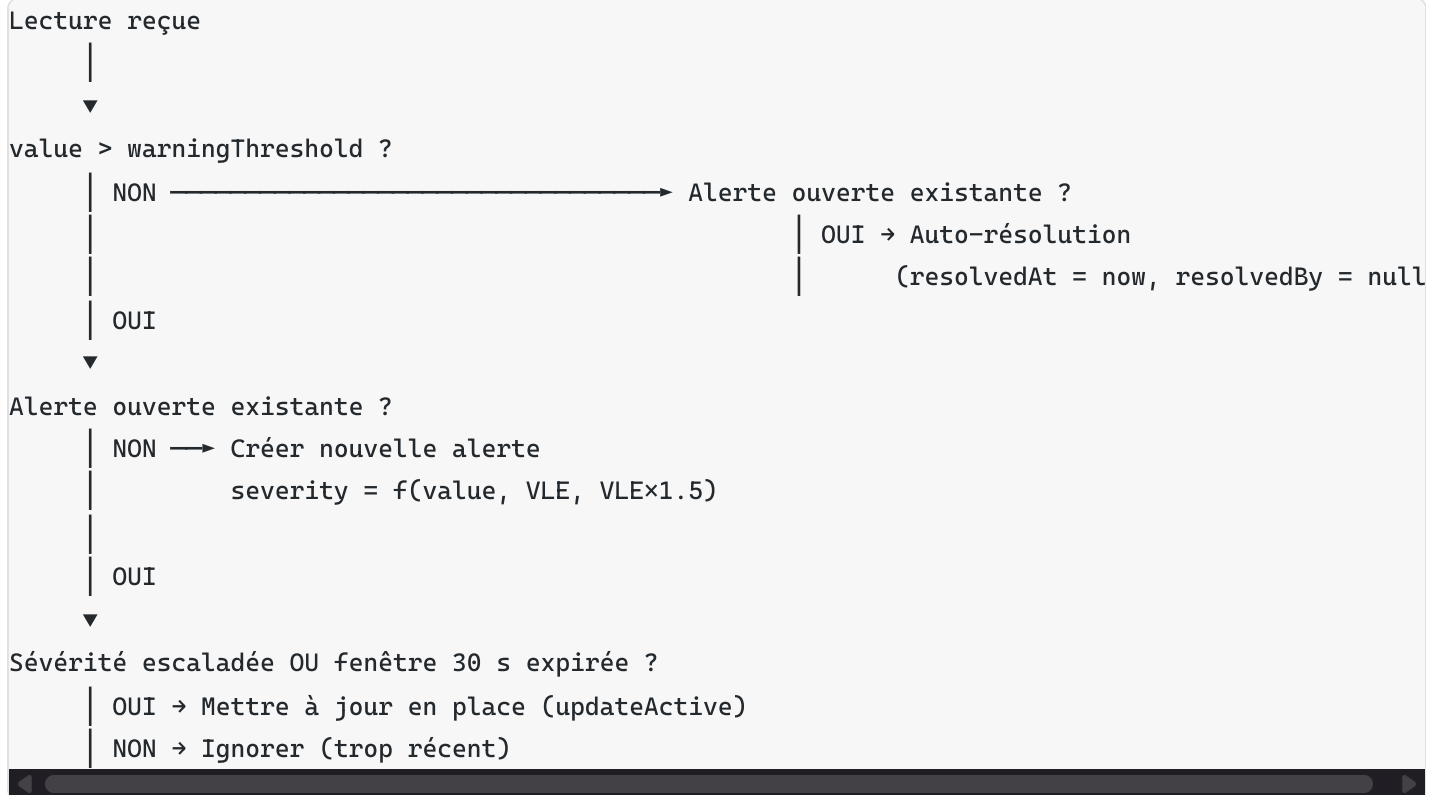
Filtres disponibles :

Polluant —————> Sélection mono ou multi-polluant
 Période — hour | day | week | month | year
 Site —————> `selectionStore.siteId` (Zustand)
 Zone —————> `selectionStore.zoneId` (Zustand)

L’état de sélection (site, zone, période) est partagé globalement via Zustand (`selectionStore`). Toutes les pages — Overview, History, Compliance, Alerts — lisent ce store et filtrent leurs données en conséquence. Un changement de zone sur la page History se répercute instantanément sur la page Overview.

III.5.3 Module Alertes

Conception du moteur d’alertes. Le moteur garantit **une seule alerte active par couple (capteur × polluant)**, évitant la génération de milliers de doublons sur une période de dépassement continu.



Niveaux de sévérité :

Niveau	Condition	Signification
Warning	value > warningThreshold (80% VLE)	Approche du seuil
High	value > VLE	Dépassement réglementaire
Critical	value > VLE × 1,5	Dépassement grave

États d'une alerte. L'acquiescement (`isAcknowledged`) et la résolution (`resolvedAt`) sont deux états **indépendants** : acquiescer signifie "j'ai vu cette alerte", résoudre signifie "le problème est corrigé". La résolution automatique (par descente sous le seuil) ne force pas l'acquiescement, ce qui permet à l'opérateur de retrouver dans l'historique les alertes résolues sans intervention humaine.

III.5.4 Module Génération de Rapports

La génération de rapport est un processus **asynchrone déclenché à la demande** par le superviseur ou l'auditeur.

Conception du flux de génération :



Le modèle `Report` stocke `pollutantScores` comme une `Map<String, Number>` MongoDB, permettant un nombre variable de polluants sans contrainte de schéma. Le champ `status` (`DRAFT/SUBMITTED/APPROVED`) prévoit un workflow de validation réglementaire future pour la soumission à l'ANPE.

III.5.5 Module Conformité Réglementaire

Le module conformité agrège les données de plusieurs KPIs pour présenter une vue synthétique du respect du Décret 2010-2519 par polluant et par zone.

Conception des indicateurs affichés :

Pour chaque polluant P :

Statut conformité = f(C_{moy}, VLE)

CONFORME si C_{moy} ≤ VLE

EN LIMITE si 0,8×VLE < C_{moy} ≤ VLE

NON CONFORME si C_{moy} > VLE

Taux de dépassement = (N_{breach} / N_{total}) × 100

Tendance = TD(période_actuelle) vs TD(période_précédente)

Le backend fournit cet agrégat via `GET /api/thresholds/report` qui croise la collection `Polluant` (valeurs `regulatoryLimit`) avec les données agrégées de `AggregateData`. La page Compliance du frontend affiche ces résultats avec un code couleur (vert/jaune/rouge) et permet l'export CSV ou PDF.

III.5.6 Module KPIs

Quatre indicateurs de performance environnementale sont calculés par `KPIService` selon les formules définies en accord avec les pratiques de mesure industrielle tunisiennes.

KPI 1 — Taux de Dépassement (TD)

$$TD = \frac{N_{breach}}{N_{total}} \times 100 \quad \text{Objectif : } \leq 2$$

$$TD = \frac{N_{breach}}{N_{total}} \times 100 \quad \text{Objectif : } \leq 2$$

KPI 2 — Émission Massique Journalière (EMJ)

$$EMJ = C_{moy} \times Q_{air} \times 86400 \times 10^{-6} \quad (\text{kg/jour})$$

$$EMJ = C_{moy} \times Q_{air} \times 86400 \times 10^{-6} \quad (\text{kg/jour})$$

Où Q_{air} est le débit volumique de la source (Nm³/s), configurable par `SiteConfig.airflow`.

KPI 3 — Indice de Performance Environnementale (IPE)

$$IPE = \frac{\sum_p w_p \times score_p}{\sum_p w_p} \times 100 \quad \text{Objectif : } \geq 95$$

$$IPE = \frac{\sum_p w_p \times score_p}{\sum_p w_p} \times 100 \quad \text{Objectif : } \geq 95$$

Avec :

- $score_p = 1$ si $C_{moy} \leq VLE$
- $score_p = \max\left(0, 1 - \frac{C_{moy} - VLE}{VLE}\right)$ sinon
- Poids réglementaires : $NO_x = 0,30$ · $SO_2 = 0,25$ · $PM_{2.5} = 0,25$ · $COV = 0,15$ · $CO_2 = 0,05$

KPI 4 — Réduction CO₂ (RCO₂)

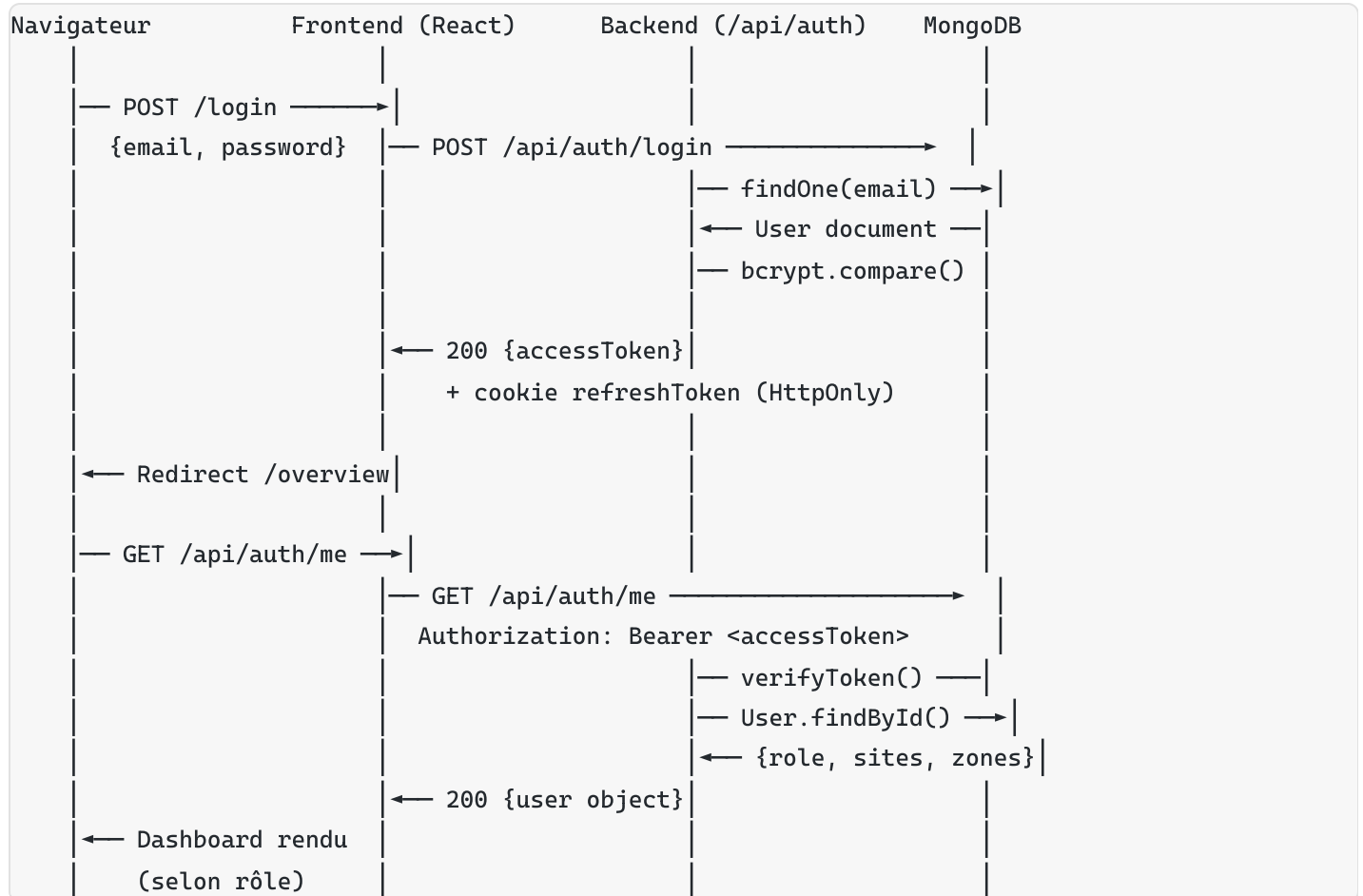
$$RCO_2 = \frac{EMJ(T) - EMJ(T_0)}{EMJ(T_0)} \times 100 \quad \text{Objectif : } \leq -5$$

$$RCO_2 = \frac{EMJ(T) - EMJ(T_0)}{EMJ(T_0)} \times 100 \quad \text{Objectif : } \leq -5$$

L'architecture KPI prévoit deux modes de calcul : calcul **à la demande** (sur lecture brute via `KPIService`) et calcul **pré-agrégé** (schedulers cron stockant les résultats dans `AggregateData`). Le frontend interroge d'abord `AggregateData` ; si aucune agrégation n'est disponible pour la période demandée, `KPIService.buildLiveDailyHistory()` recalcule sur les lectures brutes.

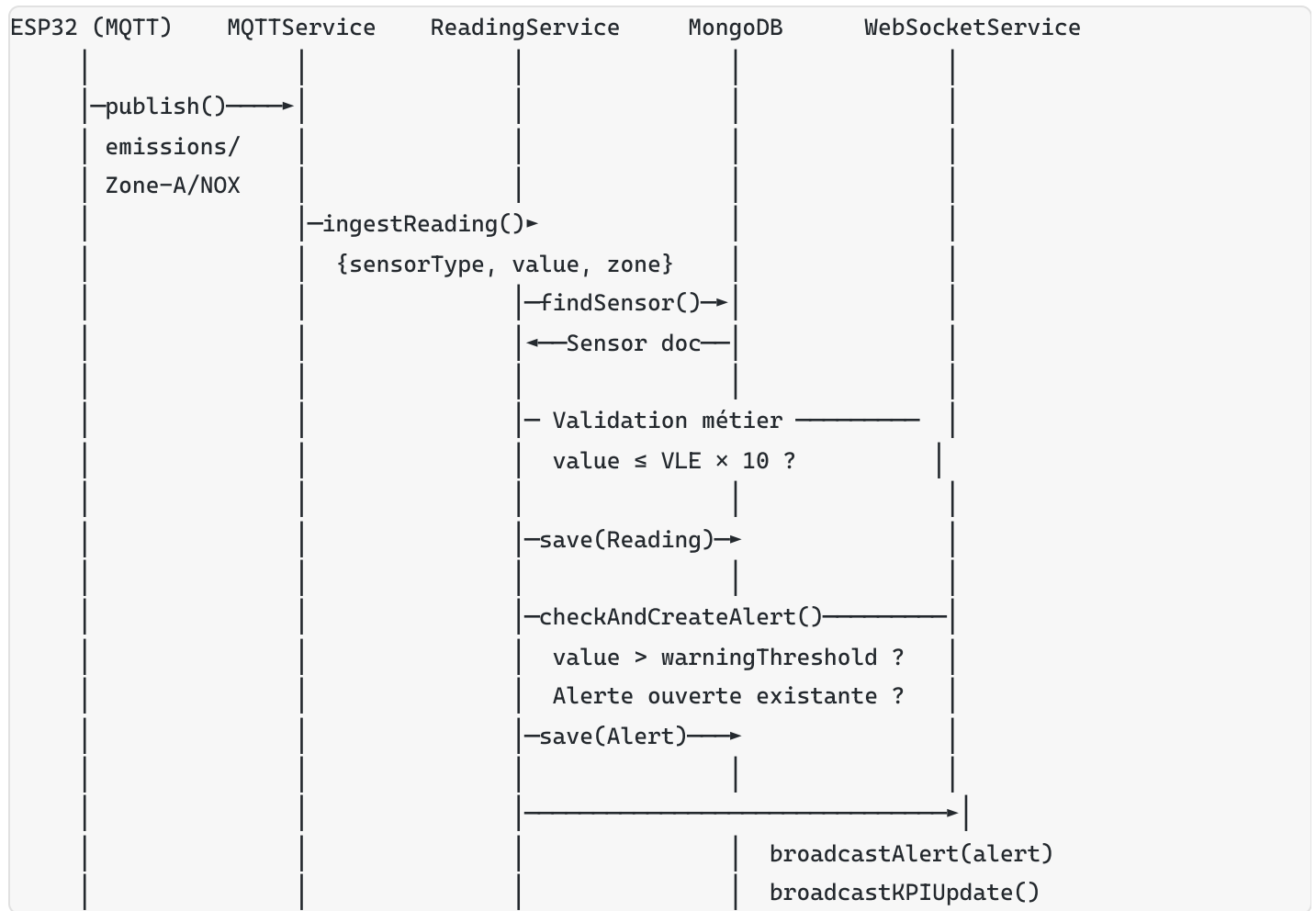
III.6 Diagrammes de séquence

III.6.1 Séquence : Authentification et accès au tableau de bord

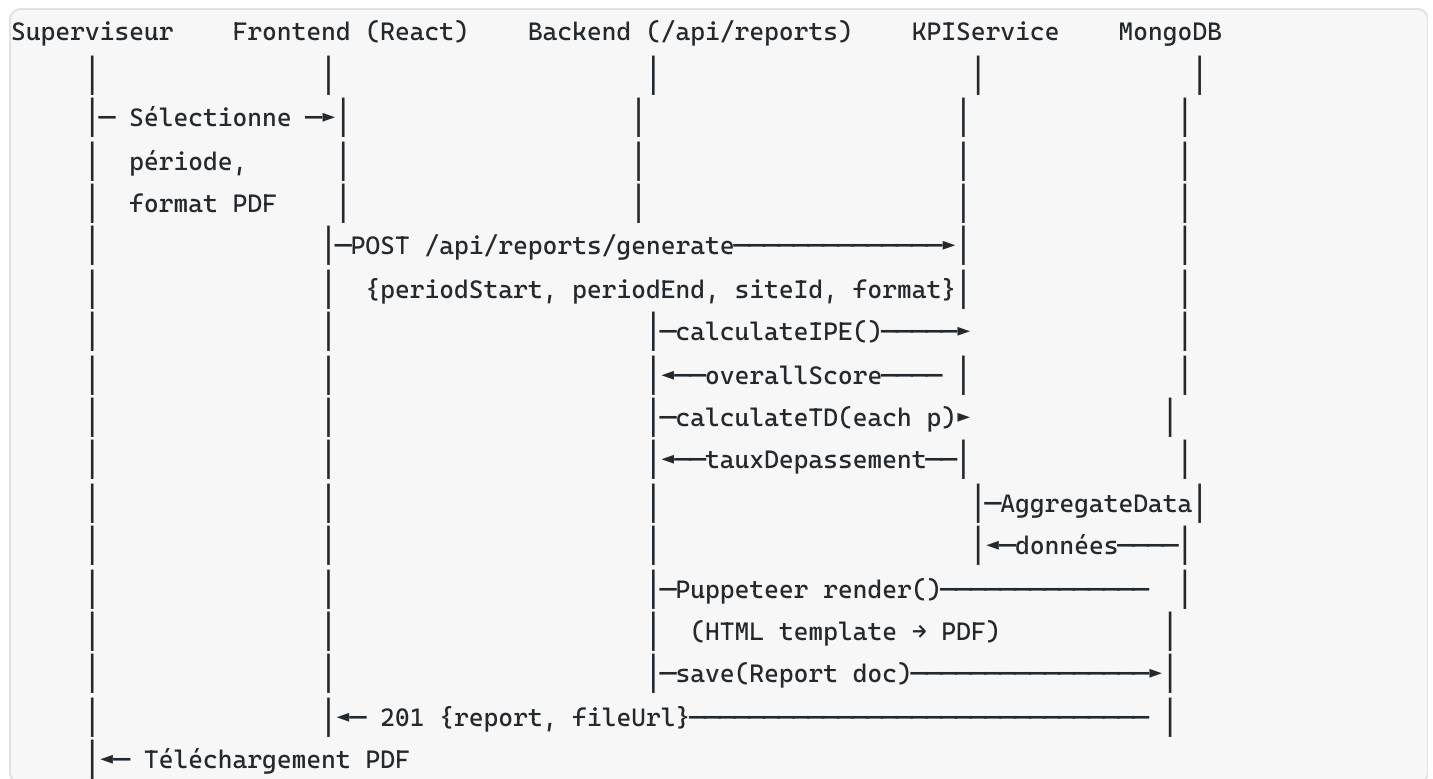


Le middleware `verifyToken` effectue une requête MongoDB sur **chaque** appel protégé pour récupérer `industryId`, `sitesManaging` et `zonesAssigned` — champs qui peuvent évoluer sans nécessiter une reconnexion. Cette décision de conception garantit que les modifications de périmètre d'un superviseur sont effectives immédiatement.

III.6.2 Séquence : Ingestion d'une lecture et déclenchement d'alerte



III.6.3 Séquence : Génération d'un rapport de conformité



III.7 Conception du contrôle d'accès (RBAC)

Le contrôle d'accès est implémenté à **trois niveaux complémentaires**, assurant une défense en profondeur :

Niveau 1 — Backend middleware. Le middleware `verifyToken` vérifie le JWT et enrichit `req.user` depuis MongoDB. Le middleware `checkRole` filtre l'accès aux routes par liste de rôles autorisés. Les alertes, lectures et rapports sont filtrés dynamiquement selon le périmètre de l'utilisateur (`industryId` , `sitesManaging` , `zonesAssigned`).

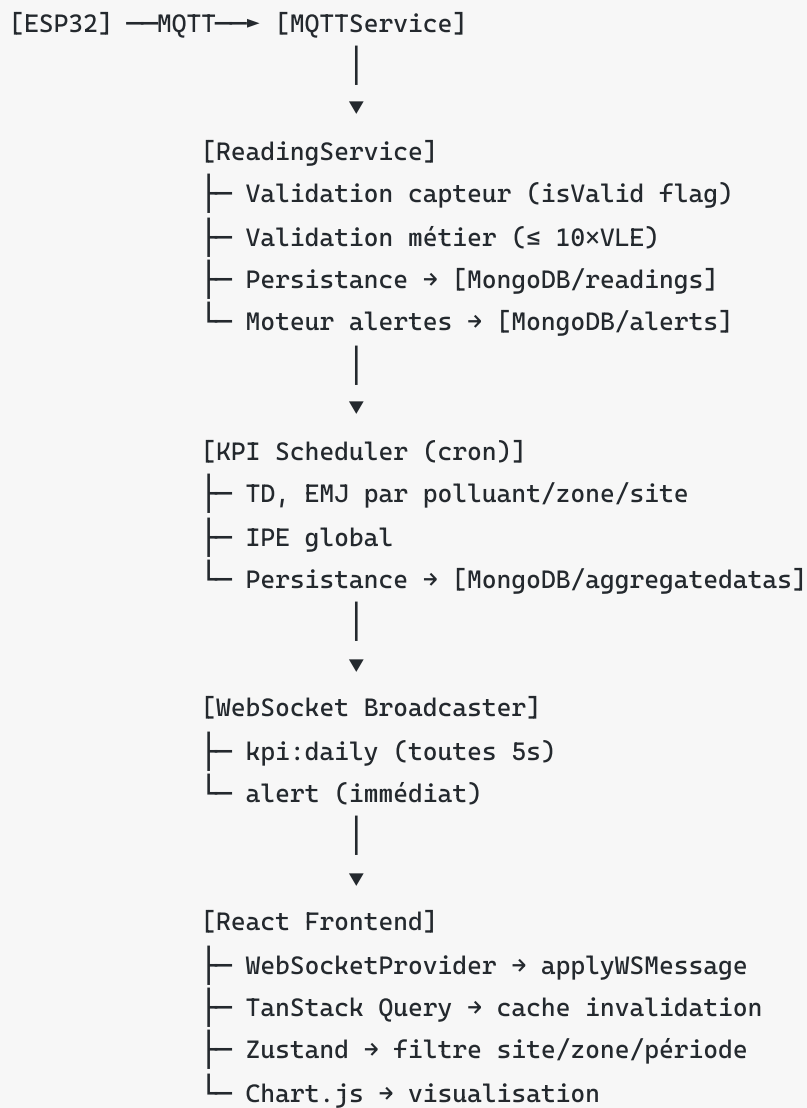
Niveau 2 — Composant ProtectedRoute (React). Chaque route du frontend vérifie les permissions via le tableau `ROLE_PERMISSIONS` avant de rendre la page. Un accès non autorisé redirige vers `/unauthorized` .

Niveau 3 — Filtrage des données. Même si un utilisateur avait accès à une page, les données retournées par l'API sont filtrées côté serveur. Un `OPERATOR` ne reçoit jamais de données hors de ses zones assignées, indépendamment de ce que le frontend affiche.

III.8 Flux de données — Vue d'ensemble de la conception

La figure suivante synthétise les flux de données de la conception end-to-end, de la collecte IoT à l'affichage dashboard, en montrant les points de décision clés.

Flux de données EmissionsIQ



III.9 Synthèse des choix de conception

Problème de conception	Choix retenu	Justification
Données temps réel	WebSocket push + TanStack Query invalidation	Évite le polling, cohérence cache garantie
Schéma capteurs hétérogènes	MongoDB documents flexibles	Pas de migration lors d'ajout de capteur
Performance KPI sur 172 k lectures	Pré-agrégation cron + dénormalisation	Réponse < 100 ms sur données historiques
Unicité des alertes	Map mémoire <code>_activeAlerts</code> + warmup	Évite la multiplication des doublons en cas de burst
Sécurité multi-tenant	RBAC 3 niveaux + fetch DB par requête	Isolation garantie même après changement de rôle
Rapports réglementaires	Puppeteer PDF + KPIService existant	Réutilisation des formules, rendu fidèle
Évolutivité seuils réglementaires	<code>ThresholdConfig</code> paramétrable par secteur	Déploiement multi-industrie sans modification du code